

AdminG

Table of contents

1 AdminG	5
Documentacion técnica AdminG	6
Anexos A-E	6
Introducción	7
1.1 Estructura documental	7
2 Anexo A	8
2.1 Requerimientos y alcance de AdminG Systems	8
2.2 Proposito del anexo	8
2.3 Alcance funcional	8
2.4 Requerimientos funcionales y no funcionales	9
2.5 Mapa de requerimientos	11
2.6 Criterios de aceptacion	11
2.7 Cierre	11
3 Anexo B	13
3.1 Arquitectura y despliegue de AdminG Systems	13
3.2 Proposito del anexo	13
3.3 Arquitectura logica	13
3.4 Modulos operativos	14
3.5 Administración y control	14
3.6 Seguridad y control de acceso	14
3.7 Despliegue en Oracle Free Tier	15
3.8 Arquitectura de despliegue	16
3.9 Ciclo de solicitud	17
3.10 Flujo de datos y persistencia	18
3.11 Decisiones arquitectonicas	18
3.12 Conexiones documentales	18
4 Anexo C	19
4.1 Modelo de datos y persistencia	19
4.2 Proposito del anexo	19
4.3 Entidades principales	19
4.4 Modelo entidad-relacion	22

4.5	Vista ejecutiva simplificada	22
4.6	Gestión de usuarios y subscripciones	23
4.6.1	Autenticación	23
4.6.2	Delegación	24
4.6.3	Capacidades por plan	24
4.7	Clientes y gestión comercial	25
4.8	Servicios y operación	26
4.9	Facturación y caja	27
4.9.1	Factura	28
4.9.2	Pagos	29
4.9.3	Caja	30
4.10	Inventario	30
4.11	Documentación y auditoría	32
4.12	Normalizacion y consistencia	32
4.13	Persistencia en VM y crecimiento	33
4.14	Informacion para gestionar e introducir	33
4.15	Diccionario resumido	33
4.16	Cierre	33
5	Anexo D	35
5.1	Calidad, seguridad y gobernanza	35
5.2	Proposito del anexo	35
5.3	Estrategia de calidad	35
5.4	Seguridad aplicada	37
5.5	Pruebas realizadas	37
5.6	Gobernanza Git y documentacion	37
5.7	Riesgos y controles	39
5.8	Cierre	39
6	Anexo E	40
6.1	Implementacion, evidencia y operacion	40
6.2	Proposito del anexo	40
6.3	Flujo implementado	40
6.4	Endpoints y modulos visibles	40
6.5	Evidencias graficas	41
6.5.1	Frontend en desarrollo y producción	41
6.5.2	Despliegue operativo	43
6.6	Cierre	44
7	Anexo F	45
7.1	Firmas	45
7.1.1	Aprendiz	45
7.1.2	Ente encargado	45

7.1.3 Instructor Sena de seguimiento 45

1 AdminG

Documentacion técnica AdminG

Anexos A-E

La documentación técnica constituye el conjunto de registros, especificaciones, procedimientos y criterios que permiten comprender, operar, mantener y desarrollar un sistema de manera consistente y trazable. Su finalidad es preservar el conocimiento institucional del proyecto, facilitar la continuidad operativa y establecer un marco de referencia para la toma de decisiones técnicas presentes y futuras.

Introducción

Esta documentación consolida los anexos técnicos, funcionales y operativos de AdminG. AdminG es una plataforma de gestión concebida como parte del ecosistema Eduardo Martinez Idealism, EMI(Ecosistema de Mediación idealista) ERP-CRM, EOE(Ecosistema operativo estratégico), un conjunto de iniciativas orientadas a la organización del conocimiento, la administración de recursos y el desarrollo de soluciones tecnológicas con enfoque sistémico. En este contexto, AdminG actúa como una herramienta destinada a centralizar procesos administrativos y tecnológicos mediante una estructura escalable, documentada y sostenible en el tiempo.

1.1 Estructura documental

- Anexo A. Requerimientos y alcance
- Anexo B. Arquitectura y despliegue
- Anexo C. Modelo de datos y persistencia
- Anexo D. Calidad, seguridad y gobernanza
- Anexo E. implementación, evidencia y operación
- Anexo F. Firmas

2 Anexo A

2.1 Requerimientos y alcance de AdminG Systems

Eduardo Martínez

2.2 Proposito del anexo

Este anexo organiza los requerimientos funcionales y no funcionales de **AdminG Systems**, una plataforma de administracion general orientada a pequenas, medianas y grandes empresas, publicas, privadas y mixtas. Su funcion dentro del conjunto documental es definir que necesita resolver el sistema antes de explicar como se estructura, como persiste la informacion, como se valida su calidad y como se implementa en un entorno real. Por esa razon, este documento se conecta directamente con el Chapter 3, donde se traduce el alcance en arquitectura, con el Chapter 4, donde se concreta el modelo de datos, con el Chapter 5, donde se verifican criterios de calidad, y con el Chapter 6, donde se documenta la evidencia de implementacion y despliegue en admining-systems.online.

AdminG combina autenticacion, gestion de usuarios, clientes, citas, servicios, pagos, inventario, reportes, documentos, autorizaciones, CRM, onboarding y control de planes. El objetivo no es construir un modulo aislado, sino una base SaaS progresiva: una aplicacion que pueda empezar con operaciones pequenas y crecer hacia control financiero, trazabilidad documental, gestion de equipos y reglas de negocio por plan.

2.3 Alcance funcional

El sistema debe permitir registro e inicio de sesion mediante credenciales seguras, emision de tokens JWT, refresh tokens y control de acceso por usuario, rol y plan. Una vez autenticado, el usuario accede a un tablero de trabajo donde puede consultar metricas operativas, administrar clientes, programar citas, registrar servicios, manejar pagos, emitir facturas, controlar inventario y revisar reportes.

Tambien se requiere un esquema comercial flexible. AdminG no debe tratar a todos los usuarios igual: un plan gratuito o basico puede habilitar clientes y agenda, mientras que

planes superiores pueden activar inventario, pagos, reportes avanzados, CRM, facturación, documentos o funciones de equipo. Esta regla exige que la autorización no dependa solo del rol, sino también del plan contratado, del estado de prueba, de los límites definidos y de las características habilitadas.

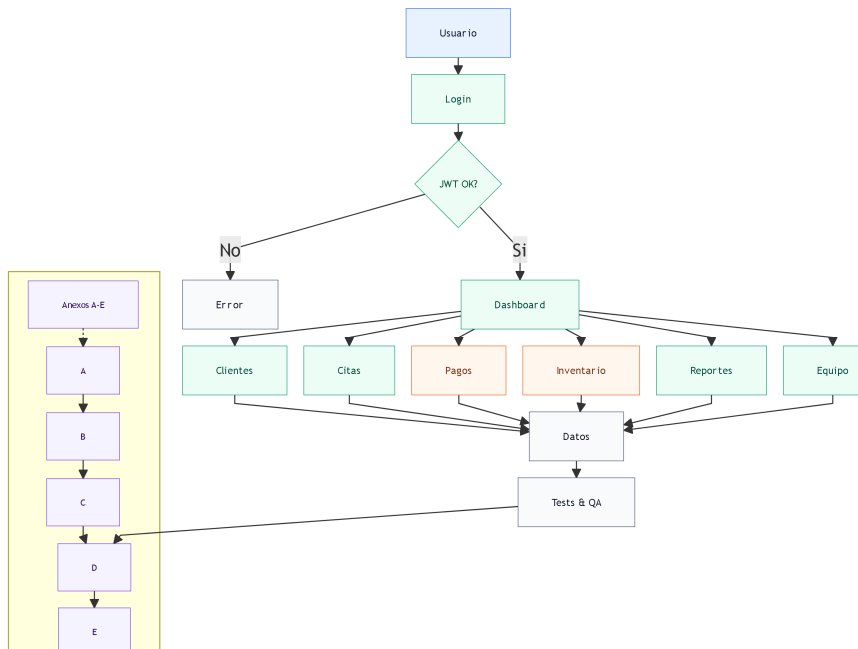
2.4 Requerimientos funcionales y no funcionales

A continuación se detallan los requerimientos con su clasificación explícita como funcional o no funcional.

#	Categoría	Tipo	Requerimiento
1	Autenticación	Funcional	Registro, login, recuperación de contraseña, JWT, refresh token, usuario activo y cierre seguro de sesión.
2	Usuarios y roles	Funcional	Administrador, manager, miembros de equipo, subusuarios, permisos y jerarquía por organización.
3	Clientes y CRM	Funcional	CRUD de clientes, seguimiento, historial, datos de contacto, mascotas o entidades relacionadas según tipo de negocio.
4	Operación	Funcional	Agenda de citas, servicios, paquetes, pagos, facturas, caja, inventario y movimientos.
5	Planes	Funcional	Catálogo Free, Basic, Plus, Start, Max o equivalentes, límites y características activables.

#	Categoria	Tipo	Requerimiento
6	Reportes	Funcional	Dashboard, metricas de clientes, citas, productos, ingresos y comportamiento operativo.
7	Documentos	Funcional	Facturas, autorizaciones, documentos trazables y posibilidad de evolucionar hacia QR o auditoria formal.
8	Seguridad	No funcional	Hash de contraseñas, CORS estricto, secretos fuera del código, validación Pydantic y control de acceso por recurso.
9	Disponibilidad	No funcional	Despliegue en VM Ubuntu con backend FastAPI sirviendo API y frontend estatico.
10	Mantenibilidad	No funcional	Arquitectura modular, pruebas automatizadas, migraciones, documentación viva y ramas Git controladas.

2.5 Mapa de requerimientos



2.6 Criterios de aceptacion

Un requerimiento se considera aceptado cuando existe una ruta o vista que lo soporte, un modelo o esquema que lo represente, una regla de permisos que limite su uso si aplica y una evidencia documental que lo conecte con los anexos. Por ejemplo, el login requiere pantalla visible, endpoint de autenticacion, hash de contraseña, token emitido, usuario persistido y evidencia de acceso al dashboard. El inventario requiere categorías, items, cantidades, movimientos, validaciones y reglas por plan.

El despliegue en Oracle Free Tier agrega un requerimiento operativo: la aplicación debe poder ejecutarse en una VM Micro Ubuntu con recursos limitados, por lo que el diseño debe ser prudente con memoria, procesos, logs, base de datos y dependencias. Para desarrollo en producción controlada, SQLite puede ser aceptable como etapa inicial si se acompaña de backups, pero el crecimiento natural del sistema debe contemplar PostgreSQL.

2.7 Cierre

El Anexo A define a AdminG como una plataforma modular, segura y escalable. Su valor está en integrar funciones administrativas que suelen estar separadas: clientes, agenda, pagos,

inventario, reportes y documentos. La evidencia visual del login y dashboard demuestra que el alcance no es solo teorico; ya existe una interfaz desplegada y operable. Los anexos siguientes explican como esa necesidad se convierte en arquitectura, datos, calidad e implementacion.

3 Anexo B

3.1 Arquitectura y despliegue de AdminG Systems

Eduardo Martínez

3.2 Proposito del anexo

Este anexo documenta la arquitectura de **AdminG Systems** y su despliegue en una VM Micro Ubuntu de Oracle Cloud Free Tier. Mientras el Chapter 2 define los requerimientos, este documento explica como se organizan los componentes para cumplirlos: frontend, backend, base de datos, seguridad, modulos de negocio, archivos estaticos, configuracion de entorno y publicacion.

La arquitectura actual combina **FastAPI** como backend, **SQLAlchemy** como capa ORM, **Alembic** para migraciones, **Pydantic** para validacion, routers modulares por dominio y un frontend distribuido desde **frontend-dist**. En desarrollo local se accede por `http://localhost:8000/`; en el despliegue mostrado se accede por la IP publica `http://157.151.206.208`. Esta dualidad es importante: permite validar cambios localmente y luego observar la misma experiencia en la VM. Se continua el proceso con dominio `adming-systems.online` y encriptado.

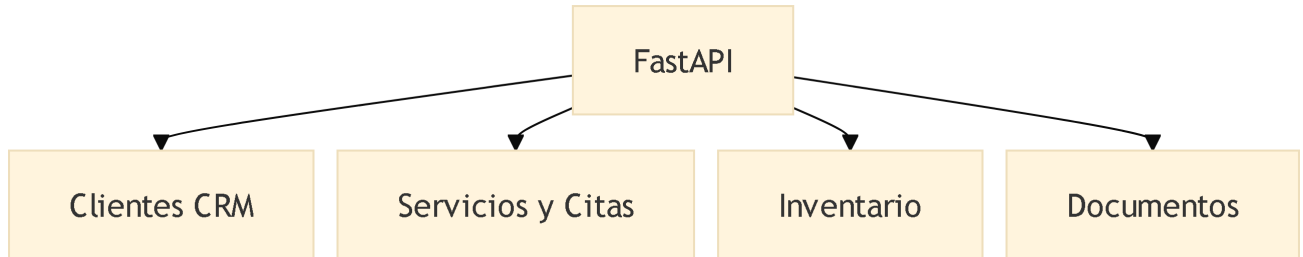
3.3 Arquitectura logica

AdminG usa una arquitectura por capas con separacion clara de responsabilidades. La capa de presentacion renderiza las vistas, administra la sesion del usuario y consume la API. La capa de aplicacion recibe solicitudes HTTP, valida tokens, aplica reglas de plan y ejecuta casos de uso. La capa de persistencia guarda usuarios, clientes, citas, pagos, inventario, facturas, documentos y reglas comerciales.

Los routers principales incluyen autenticacion, usuarios, clientes, mascotas, negocio, citas, servicios, planes, inventario, pagos, caja, facturas, notificaciones, documentos, autorizaciones, CRM, onboarding e identidad. Tambien existen routers opcionales para tesoreria, ensamble, proyectos, inventario JAC, estrategia, reportes, administracion, inteligencia artificial, operaciones y EOE, cargados por feature flags o disponibilidad de modulo.

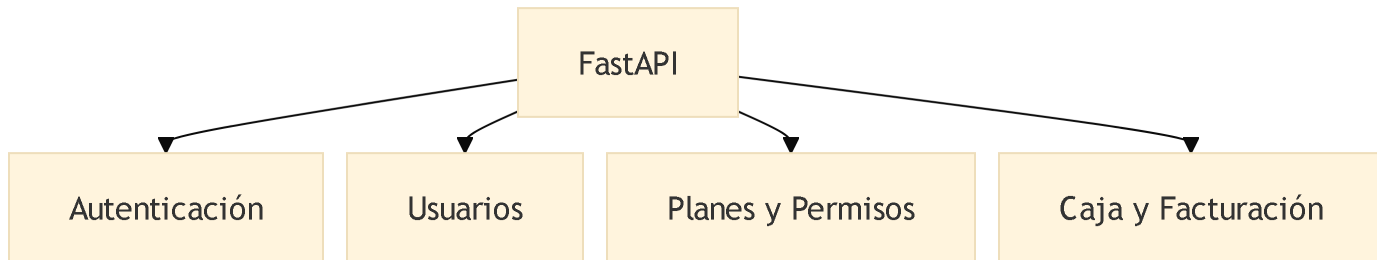
3.4 Módulos operativos

Los módulos operativos soportan la gestión diaria del negocio, incluyendo atención de clientes, programación de servicios y control de inventario



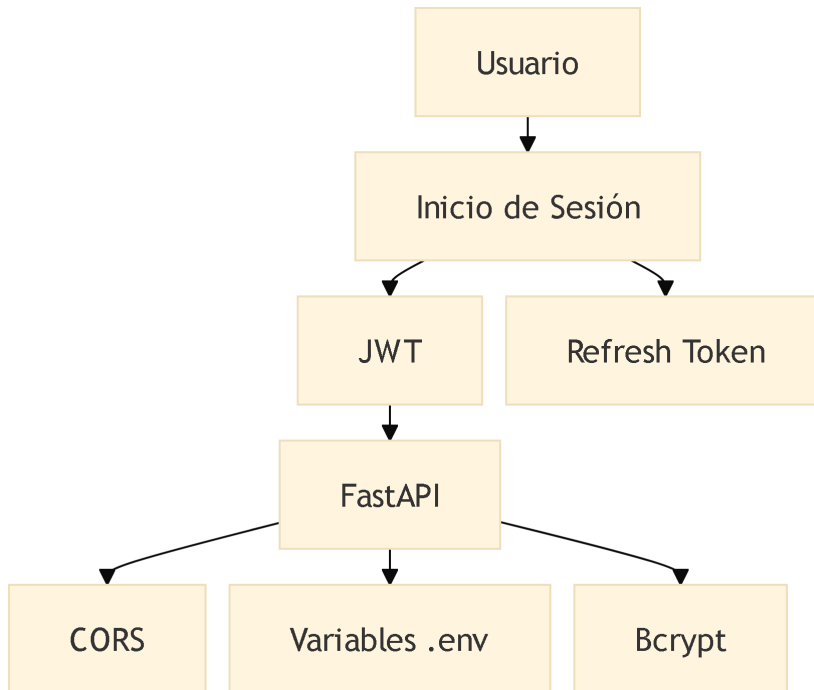
3.5 Administración y control

Los componentes administrativos gestionan la autenticación, los usuarios, los planes contratados y las operaciones financieras de la plataforma.



3.6 Seguridad y control de acceso

La seguridad se implementa mediante autenticación basada en JWT, control de acceso mediante permisos, protección criptográfica de credenciales y configuración segura mediante variables de entorno.

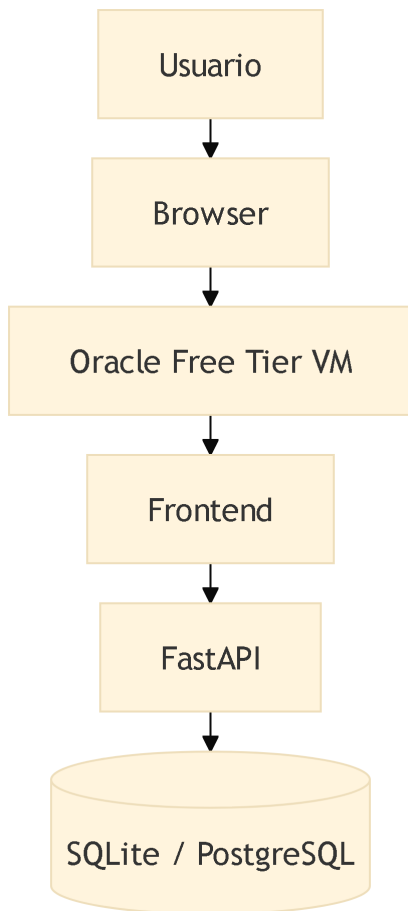


3.7 Despliegue en Oracle Free Tier

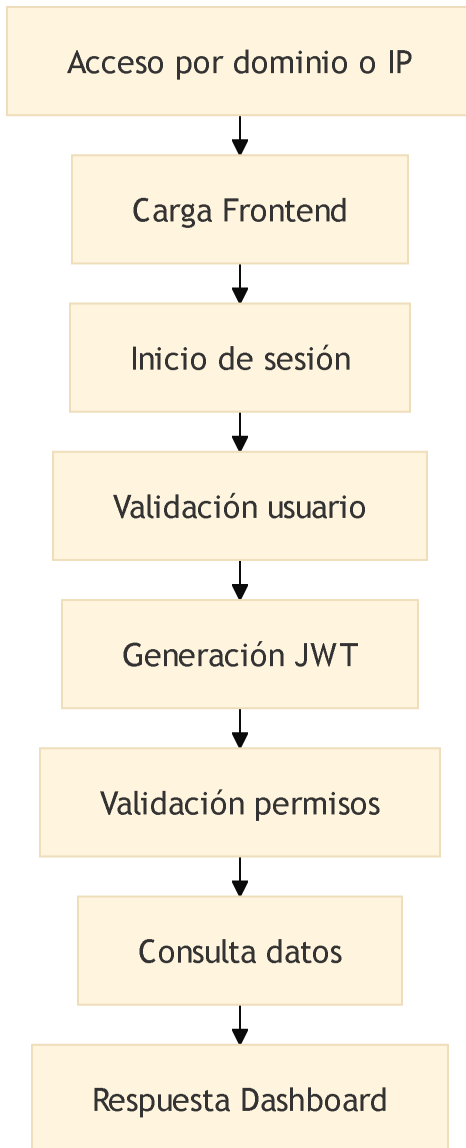
El despliegue en Oracle Free Tier es adecuado para desarrollo en producción controlada porque ofrece una VM siempre gratuita, acceso SSH, disco persistente y libertad para ejecutar FastAPI como proceso del sistema. Frente a plataformas serverless, esta opción permite conservar SQLite temporalmente, servir el frontend estático desde la misma aplicación y controlar configuraciones de red, puertos, firewall y logs.

La VM debe considerarse un entorno productivo de baja escala, no un simple laboratorio. Por ello, aunque la captura muestre **Not secure**, el cierre técnico debe incluir HTTPS, dominio, CORS restringido, variables de entorno reales, secretos robustos, bloqueo de endpoints administrativos de desarrollo y backups de `app.db`. Si el proyecto crece o se habilita multiusuario intensivo, la persistencia debe migrarse a PostgreSQL.

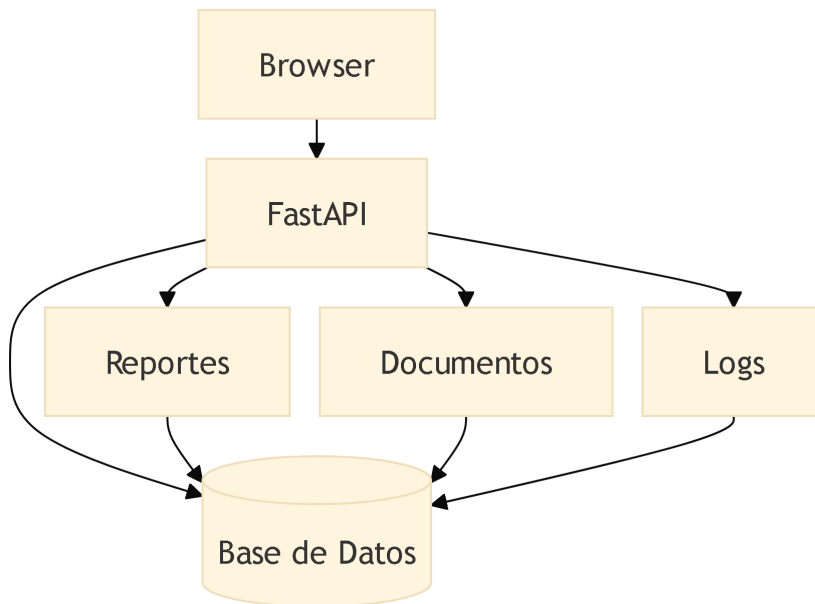
3.8 Arquitectura de despliegue



3.9 Ciclo de solicitud



3.10 Flujo de datos y persistencia



3.11 Decisiones arquitectonicas

Se eligió FastAPI por su rendimiento, tipado, documentación automática y compatibilidad con Pydantic. SQLAlchemy permite mantener independencia parcial frente al motor de base de datos y Alembic permite evolucionar esquemas. El frontend distribuido como archivos estáticos simplifica el despliegue inicial: la misma app sirve API y SPA, evitando configurar un servidor web adicional en la primera etapa.

La modularidad es el punto central. Cada dominio tiene router, schemas, modelos y reglas propias. Esto evita que autenticación, inventario, pagos o CRM se mezclen en una única capa difícil de mantener. Además, los routers opcionales permiten que el sistema crezca sin obligar a cargar todos los módulos siempre.

3.12 Conexiones documentales

La arquitectura responde a los requerimientos del [Chapter 2](#), depende del modelo de datos descrito en el [Chapter 4](#), debe verificarse con los criterios del [Chapter 5](#) y se evidencia en la implementación del [Chapter 6](#). Cualquier cambio en despliegue, dominio, HTTPS, base de datos o proceso de arranque debe actualizar este anexo y reflejarse en los demás.

4 Anexo C

4.1 Modelo de datos y persistencia

Eduardo Martínez

4.2 Proposito del anexo

Este anexo describe el modelo de datos de **AdminG Systems**, sus entidades principales, sus relaciones, las garantías de consistencia y la forma en que el almacenamiento soporta los requerimientos definidos en el Chapter 2 y la arquitectura del Chapter 3. La base actual usa SQLAlchemy y migraciones Alembic, con SQLite como persistencia inicial para desarrollo y despliegue controlado en VM. Para una operacion productiva con mayor concurrencia, se requiere migrar a PostgreSQL.

El modelo no solo guarda informacion. Tambien expresa reglas de negocio: que usuario posee clientes, que plan habilita características, que cita puede generar pagos, que factura puede relacionarse con items, que inventario registra movimientos y que autorizaciones o documentos agregan trazabilidad.

4.3 Entidades principales

La entidad **User** representa el sujeto de acceso al sistema. Contiene correo, contraseña hashada, rol, plan, estado, jerarquia de usuario padre y datos de negocio. A partir de ella se organizan clientes, pagos, inventario, configuraciones, documentos y usuarios de equipo. **RefreshToken** permite renovar sesiones sin exponer credenciales.

Customer representa la informacion de clientes. Puede conectarse con mascotas o entidades relacionadas, citas, pagos, CRM y documentos. **Appointment** administra agenda, servicios, estados y relacion con clientes. **Service** y **ServicePackage** permiten separar servicios individuales de paquetes comerciales.

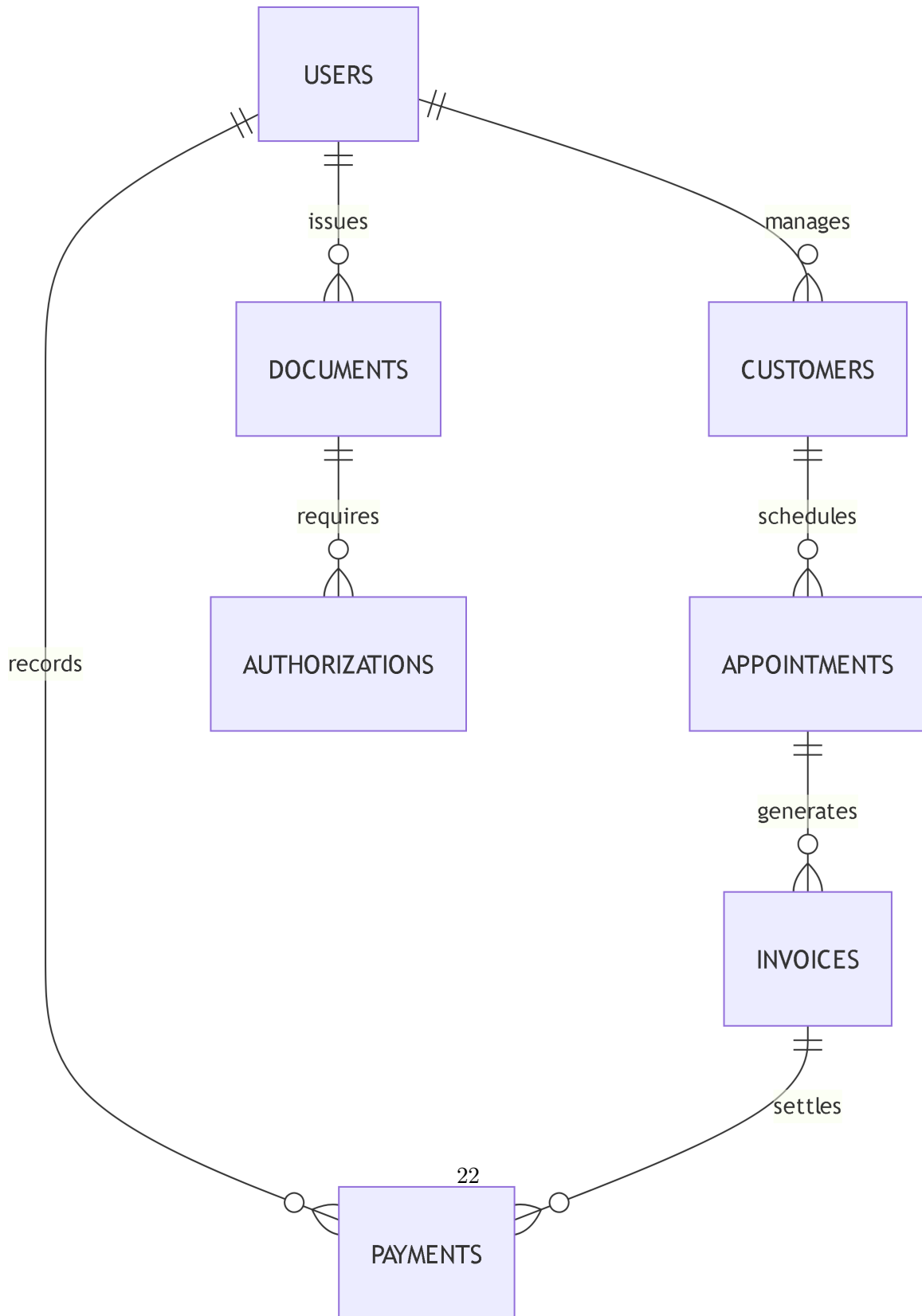
El bloque financiero esta formado por **Payment**, **PaymentItem**, **CashRegisterSession**, **CashTransaction**, **Invoice**, **InvoiceItem** y **Authorization**. Esta separacion evita que un pago sea confundido con una factura o con una sesion de caja. Una factura documenta una

obligacion o venta; un pago registra el movimiento economico; caja agrupa movimientos en una jornada; autorizaciones registran aprobaciones.

Inventario se organiza con `InventoryCategory`, `InventoryItem`, `InventoryMovement` e inventario especializado cuando aplica. Esta estructura permite controlar SKU, stock minimo, entradas, salidas y trazabilidad del producto. Planes se modela con `Plan`, `PlanLimit` y `PlanFeature`, lo que permite activar funciones sin modificar el nucleo de usuario.

4.4 Modelo entidad-relacion

4.5 Vista ejecutiva simplificada

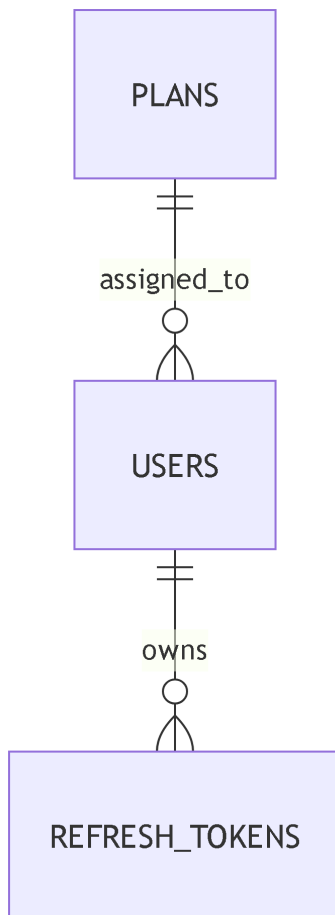


4.6 Gestión de usuarios y suscripciones

Este subconjunto modela la autenticación, delegación de usuarios y control de capacidades según el plan contratado. Constituye la base de acceso y gobierno funcional de la plataforma.

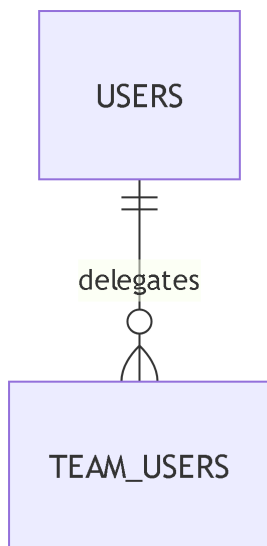
4.6.1 Autenticación

El modelo de autenticación se basa en la entidad **User**, que almacena credenciales, rol, plan y jerarquía. **RefreshToken** permite renovar sesiones sin exponer contraseñas. La relación entre **User** y **Plan** habilita o restringe características según la suscripción. La entidad **TeamUser** facilita la delegación de acceso dentro de una organización, permitiendo que un usuario principal administre subusuarios con permisos específicos. Esta estructura soporta un control de acceso granular.



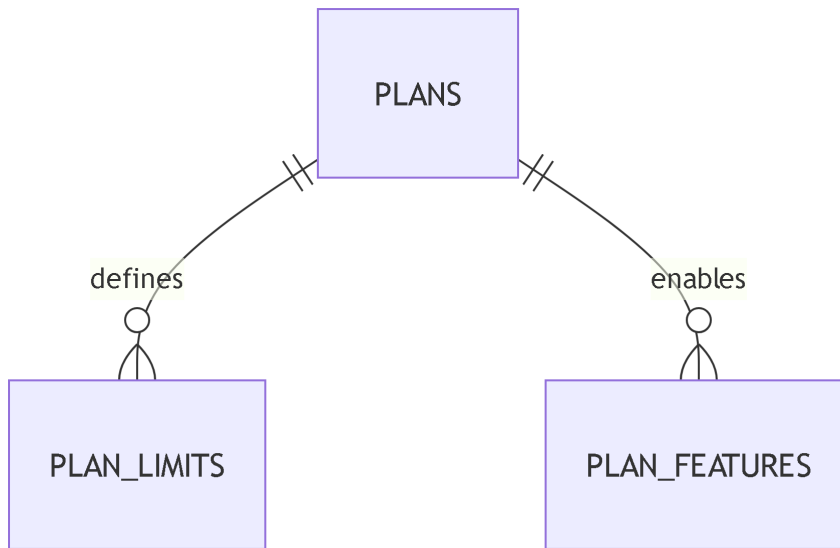
4.6.2 Delegación

La relación `TEAM_USERS` permite que un usuario principal delegue acceso a otros usuarios dentro de su organización. Esto es crucial para empresas que requieren múltiples usuarios con diferentes niveles de acceso. Por ejemplo, un gerente puede tener acceso completo, mientras que un empleado solo puede gestionar clientes o citas.



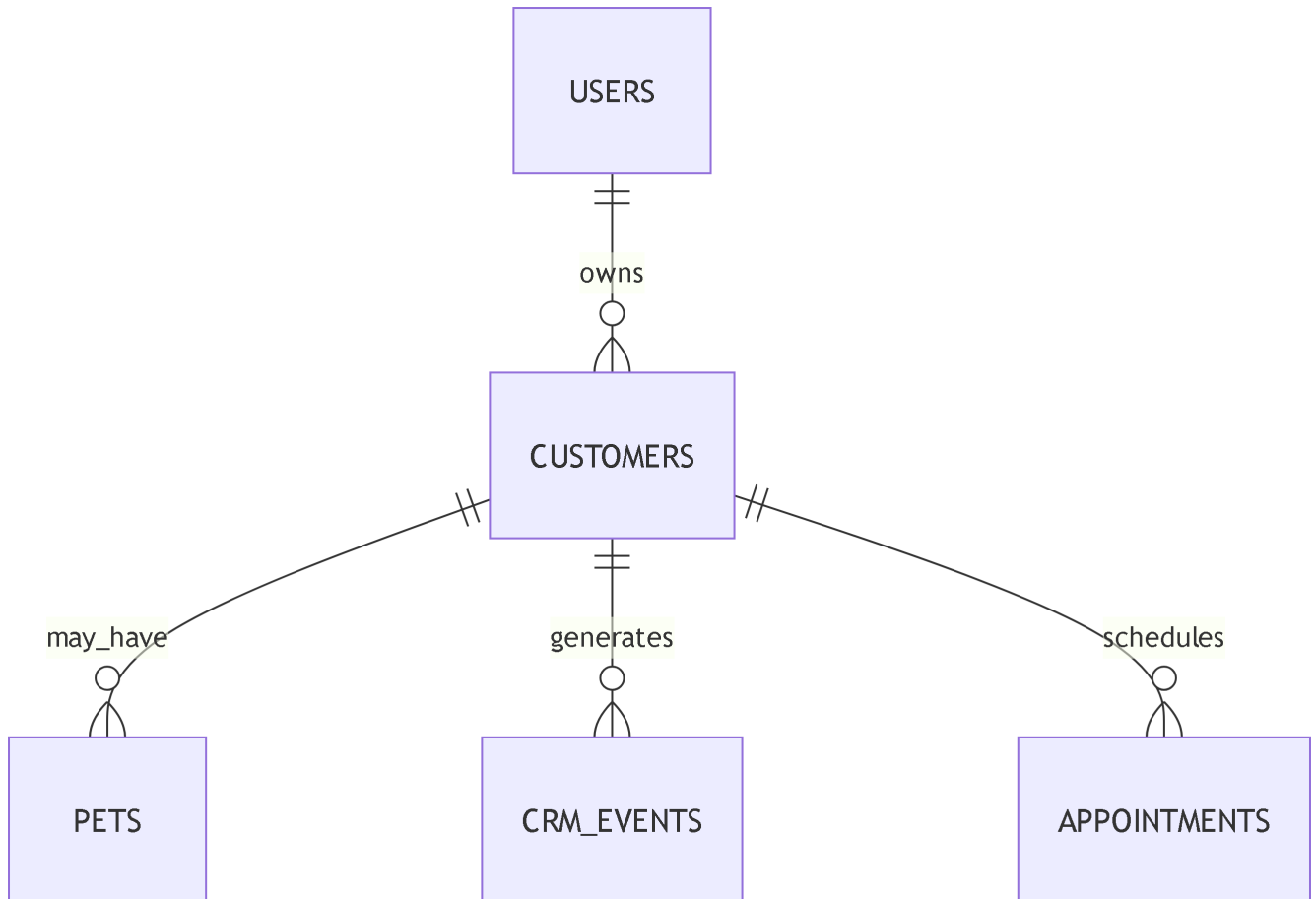
4.6.3 Capacidades por plan

La relación `PLANS` con `PLAN_LIMITS` y `PLAN_FEATURES` permite definir y habilitar características específicas según el plan contratado por cada usuario.



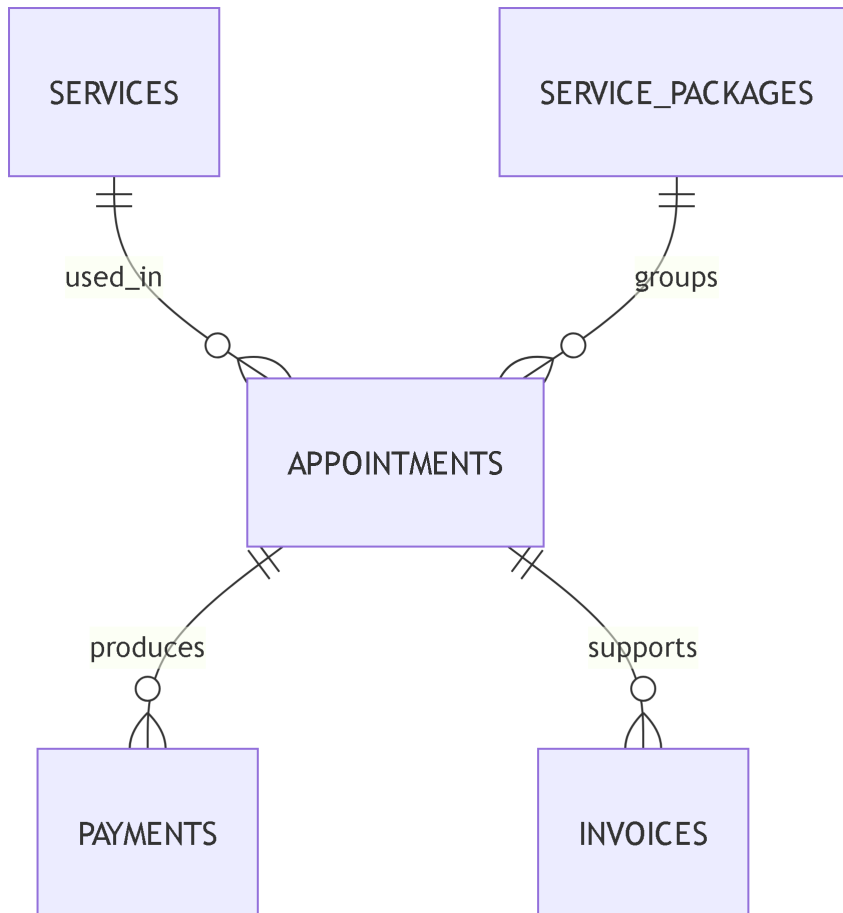
4.7 Clientes y gestión comercial

Modela la relación entre clientes, mascotas, citas y eventos de seguimiento comercial



4.8 Servicios y operación

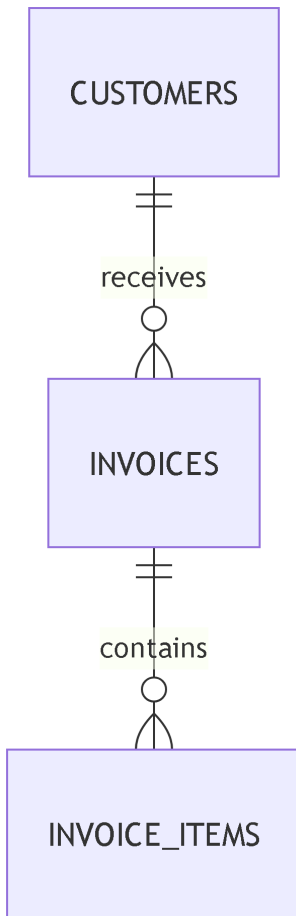
Modela la ejecución de servicios, reservas, paquetes comerciales y la generación de transacciones derivadas de la operación diaria.



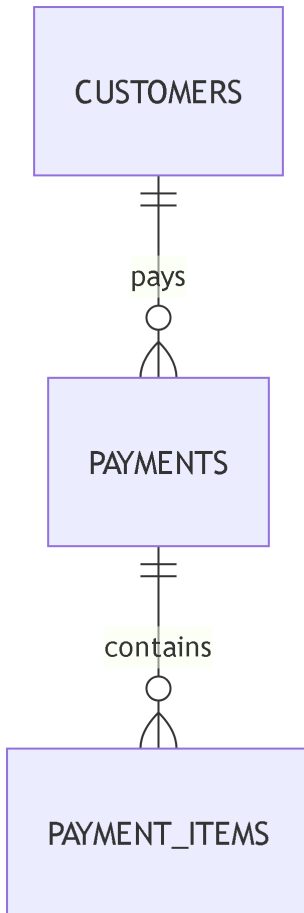
4.9 Facturación y caja

Agrupación de la lógica financiera de la plataforma, permitiendo el registro de cobros, facturación, conciliación de pagos y control de caja.

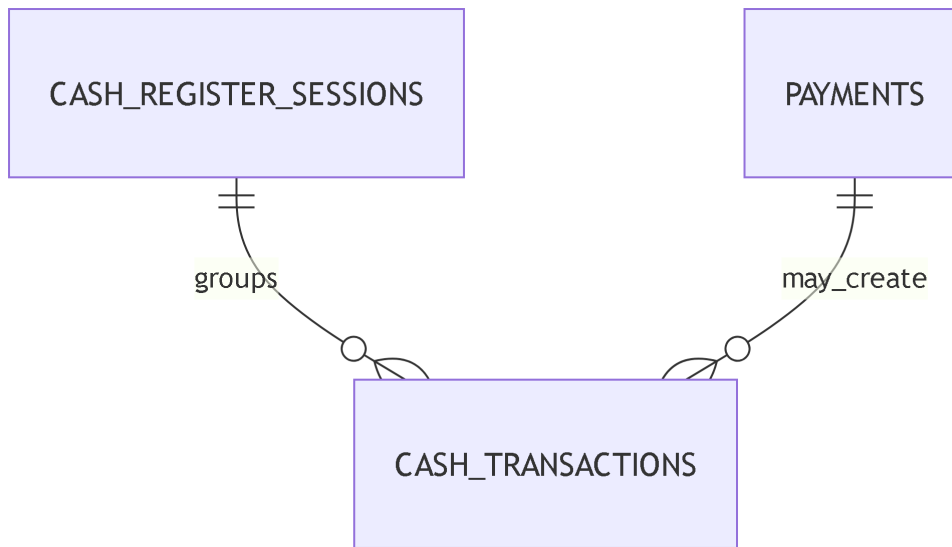
4.9.1 Factura



4.9.2 Pagos

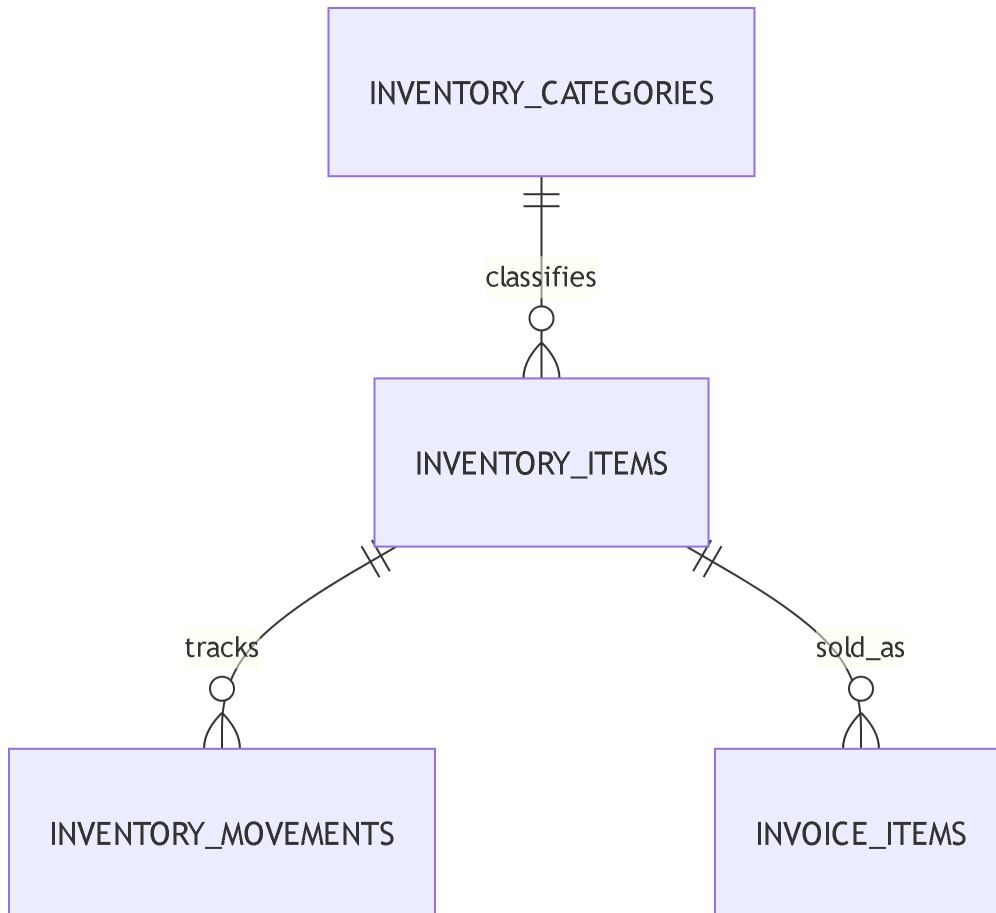


4.9.3 Caja

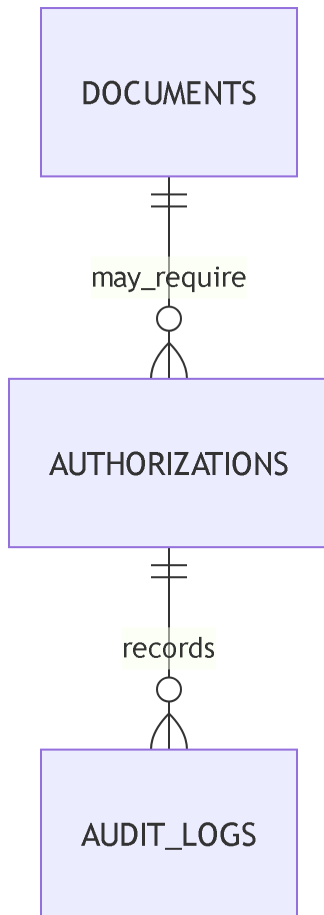


4.10 Inventario

Representa la clasificación, control y trazabilidad de los elementos inventariables



4.11 Documentación y auditoría



4.12 Normalizacion y consistencia

El modelo separa entidades con responsabilidades distintas. **Plan** no almacena todas sus reglas en una columna extensa, sino que delega límites y características a tablas relacionadas. **Payment** no debe reemplazar a **Invoice**, porque el documento legal y el movimiento de dinero tienen ciclos de vida diferentes. **InventoryMovement** evita sobrescribir la historia del stock: cada entrada o salida queda registrada.

Las garantías ACID se aplican especialmente en pagos, facturas e inventario. Si una factura descarga inventario y registra un pago, la operación debe confirmarse completa o revertirse completa. SQLAlchemy facilita esta unidad de trabajo mediante sesiones y commits controlados.

Las claves foraneas y validaciones de schemas reducen inconsistencias entre clientes, citas, pagos y usuarios.

4.13 Persistencia en VM y crecimiento

En la etapa evidenciada por las capturas, SQLite es util porque simplifica el despliegue en Oracle Free Tier. El archivo `app.db` puede vivir en disco persistente de la VM, con copias de seguridad programadas. Sin embargo, el uso de SQLite requiere prudencia: no es ideal para alta concurrencia, multiples instancias o analitica pesada. La ruta de evolucion natural es PostgreSQL, conservando SQLAlchemy y Alembic para minimizar cambios.

4.14 Informacion para gestionar e introducir

- Indicar ruta real de `app.db` en la VM Ubuntu.
- Documentar politica de backup: frecuencia, ubicacion y prueba de restauracion.
- Confirmar si se mantiene SQLite o se migra a PostgreSQL.
- Insertar captura del dashboard donde se observan clientes, citas, productos e ingresos.

4.15 Diccionario resumido

Entidad	Funcion
User	Identidad, rol, plan y jerarquia.
Customer	Cliente administrado por usuario u organizacion.
Appointment	Agenda y relacion con servicios.
Payment	Movimiento economico y estado de pago.
Invoice	Documento comercial/fiscal con items.
InventoryItem	Producto o SKU con stock.
InventoryMovement	Historial de entradas y salidas.
PlanFeature	Caracteristica habilitada por plan.
Authorization	Aprobacion formal de acciones sensibles.
AuditLog	Evidencia de cambios y trazabilidad.

4.16 Cierre

El modelo de datos de AdminG es el puente entre la arquitectura y la operacion real. Si las entidades se mantienen separadas, las reglas de negocio pueden crecer sin duplicidad. El

proximo anexo toma este modelo como base para definir pruebas, seguridad, control de cambios y criterios de calidad.

5 Anexo D

5.1 Calidad, seguridad y gobernanza

Eduardo Martínez

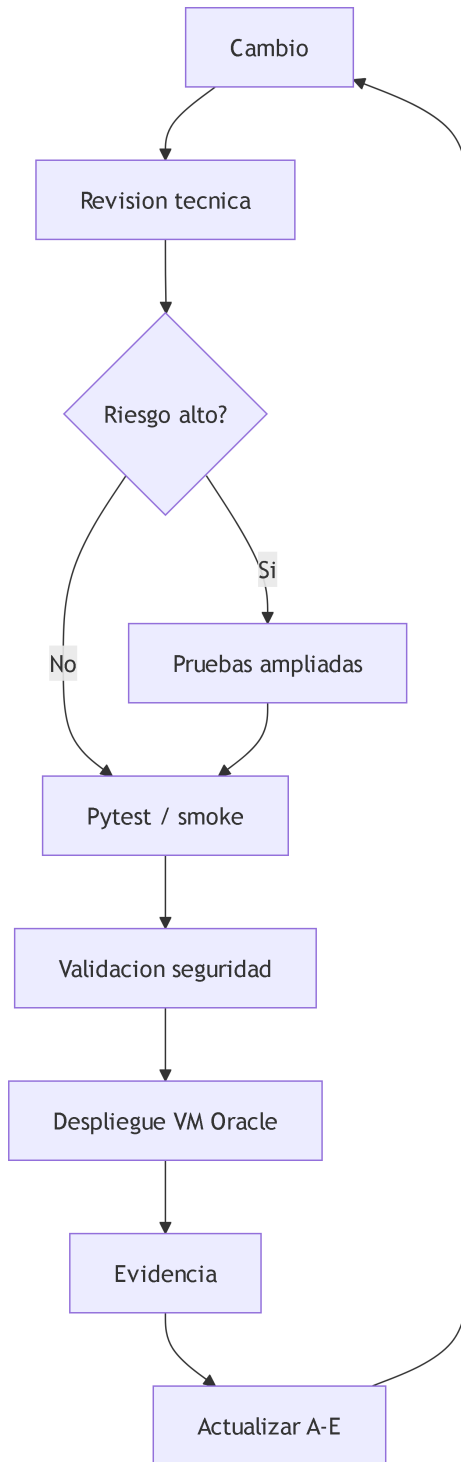
5.2 Proposito del anexo

Este anexo define la estrategia de calidad para AdminG Systems. Su funcion es verificar que los requerimientos del Chapter 2, la arquitectura del Chapter 3 y el modelo de datos del Chapter 4 se mantengan confiables cuando el sistema evoluciona. La calidad no se limita a pruebas; incluye seguridad, control de versiones, configuracion de produccion, revisiones, trazabilidad, documentacion y criterios de despliegue.

AdminG se ejecuta en un contexto sensible: administra usuarios, clientes, pagos, facturas, inventario y datos de negocio. Por ello, la calidad debe tratarse como una condicion de operacion, no como una actividad final. Cada cambio debe responder tres preguntas: que comportamiento modifica, que riesgo introduce y que evidencia demuestra que sigue funcionando.

5.3 Estrategia de calidad

La calidad se organiza en cinco frentes. El primero es **calidad de codigo**: modulos pequeños, responsabilidades separadas, nombres claros, routers por dominio, schemas de entrada y salida, y menor duplicidad posible. El segundo es **calidad funcional**: cada flujo principal debe poder probarse desde la interfaz o por API. El tercero es **calidad de datos**: transacciones, claves foraneas, migraciones y validaciones. El cuarto es **seguridad**: autenticacion, autorizacion, secretos, CORS y proteccion de endpoints. El quinto es **operacion**: despliegue reproducible, logs, backups, monitoreo minimo y rollback documentado.



5.4 Seguridad aplicada

El sistema debe validar la configuracion al iniciar en produccion. Las variables criticas incluyen `APP_ENV`, `SECRET_KEY`, politica de CORS, origen del frontend, SMTP si se usa recuperacion de contrasena y exposicion del reset token. En produccion, `CORS_ALLOW_ALL_ORIGINS` debe estar desactivado y reemplazado por una allowlist. Los secretos no deben quedar en el repositorio ni en capturas.

La autenticacion usa contrasenas hasheadas y tokens JWT. La autorizacion se apoya en rol, plan, limites y características. Esto es importante porque no basta con saber quien es el usuario; tambien se debe saber que puede hacer segun su suscripcion, su estado de prueba y su posicion dentro de la organizacion. Los endpoints administrativos o de reset de base de datos deben permanecer deshabilitados o estrictamente protegidos en entornos publicos.

5.5 Pruebas realizadas

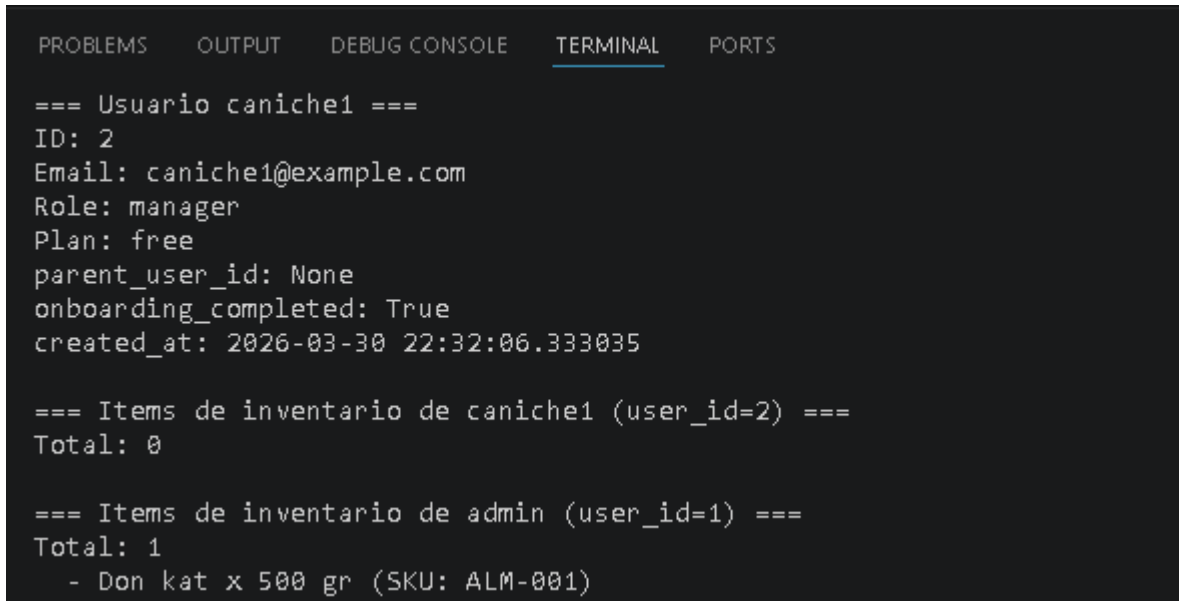
Area	Prueba
Autenticacion	Registro, login, refresh token, usuario inactivo, contrasena incorrecta.
Planes	Acceso permitido y denegado segun <code>PlanFeature</code> y <code>PlanLimit</code> .
Clientes	CRUD, aislamiento por usuario, busqueda y validaciones.
Citas	Crear, editar, cancelar, relacionar con cliente y servicio.
Pagos	Registro, estados, montos, relacion con factura o caja.
Inventario	Crear SKU, stock minimo, entrada, salida y bloqueo por plan.
Reportes	Metricas del dashboard y filtros de fecha.
Despliegue	<code>GET /health</code> , carga del login, carga del dashboard autenticado.

5.6 Gobernanza Git y documentacion

La documentacion A-E debe vivir como evidencia del avance. Cada anexo tiene una responsabilidad: requerimientos, arquitectura, datos, calidad e implementacion. Cuando un cambio modifica una regla funcional, se actualiza Anexo A; si modifica infraestructura, Anexo B; si

cambia modelos o migraciones, Anexo C; si cambia pruebas, seguridad o criterios de aceptacion, Anexo D; si cambia endpoints, capturas o despliegue, Anexo E.

Se realizaron ramas tematicas como docs/anexos-a-e, feat/payments-trial-qr, feature/inventario o sync-prod. Los commits deben ser descriptivos: docs: organize annexes a-e, feat: Vet feature and password, fix: Frontend bussiness details.

A screenshot of a terminal window with a dark background and light-colored text. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal output shows three sections of data. The first section is titled '=== Usuario caniche1 ===' and lists user details: ID: 2, Email: caniche1@example.com, Role: manager, Plan: free, parent_user_id: None, onboarding_completed: True, and created_at: 2026-03-30 22:32:06.333035. The second section is titled '=== Items de inventario de caniche1 (user_id=2) ===' and shows 'Total: 0'. The third section is titled '=== Items de inventario de admin (user_id=1) ===' and shows 'Total: 1' followed by a list item '- Don kat x 500 gr (SKU: ALM-001)'.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=== Usuario caniche1 ===
ID: 2
Email: caniche1@example.com
Role: manager
Plan: free
parent_user_id: None
onboarding_completed: True
created_at: 2026-03-30 22:32:06.333035

=== Items de inventario de caniche1 (user_id=2) ===
Total: 0

=== Items de inventario de admin (user_id=1) ===
Total: 1
  - Don kat x 500 gr (SKU: ALM-001)
```

Figure 5.1: Pruebas de Usuario

- Configurado UFW, firewall de Oracle, Nginx, HTTPS o systemd.

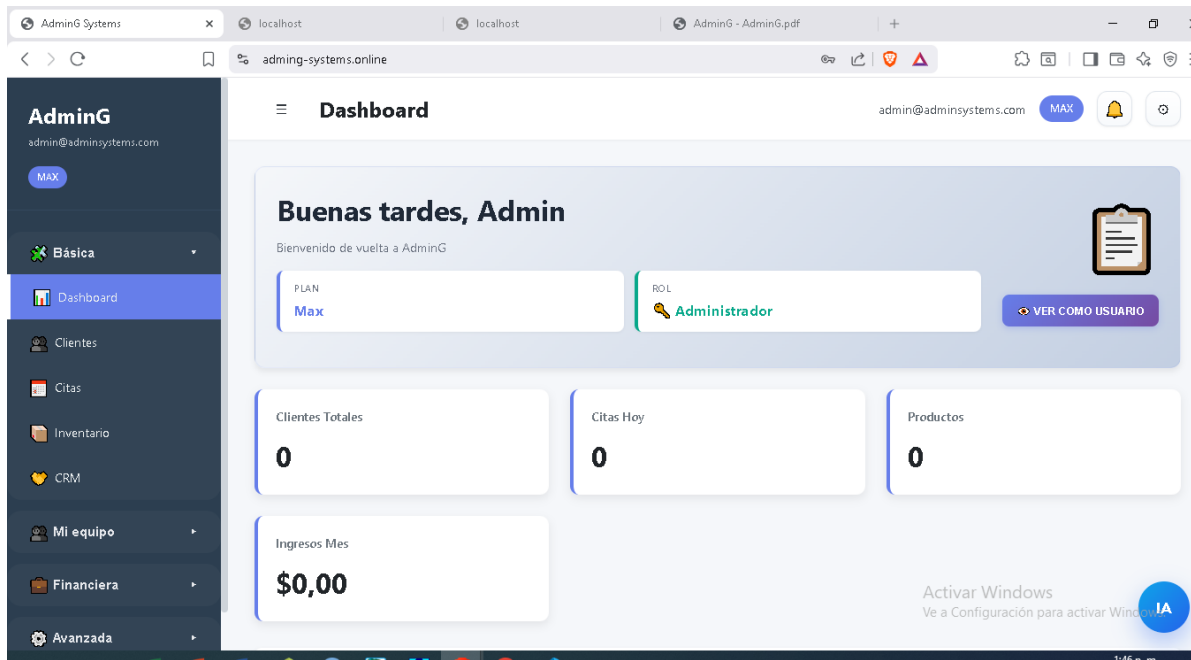


Figure 5.2: Acceso a Dashboard

5.7 Riesgos y controles

El primer riesgo es exponer la aplicación por HTTP sin TLS. El control recomendado es dominio, certificado HTTPS y redirección. El segundo riesgo es usar SQLite sin backups. El control es copia programada y migración futura a PostgreSQL. El tercero es dejar configuraciones de desarrollo en producción, como CORS abierto, reset token expuesto o endpoint de reset de base de datos. El control es validación de runtime y checklist de despliegue.

5.8 Cierre

La calidad de AdminG depende de mantener alineados código, datos, despliegue y documentación. Este anexo establece el marco para revisar cada entrega y conecta directamente con el Chapter 6, donde se presenta la implementación y la evidencia de operación.

6 Anexo E

6.1 Implementacion, evidencia y operacion

Eduardo Martínez

6.2 Proposito del anexo

Este anexo documenta la implementacion de **AdminG Systems** y reúne la evidencia practica del sistema en ejecucion. Cierra la cadena iniciada por el Chapter 2, que define requerimientos; continuada por el Chapter 3, que define arquitectura; sustentada por el Chapter 4, que define persistencia; y validada por el Chapter 5, que define calidad.

La implementacion actual integra backend FastAPI, frontend estatico, routers modulares, base de datos relacional mediante SQLAlchemy, migraciones, semillas iniciales, control de planes y despliegue en Oracle Free Tier VM Micro Ubuntu. Las capturas muestran que existe una pantalla de login disponible por IP publica, una version local en VS Code y un dashboard autenticado con usuario de prueba.

6.3 Flujo implementado

El flujo base inicia con la carga del frontend, continua con autenticacion, emision de token y consulta de datos del dashboard. El backend crea o verifica tablas al iniciar, ejecuta migraciones SQLite de arranque cuando aplica, siembra planes y tipos de negocio, y garantiza la existencia de un usuario administrador por defecto para tareas iniciales. La aplicacion monta archivos estaticos de `frontend-dist` y responde rutas SPA desde `index.html`.

6.4 Endpoints y modulos visibles

Los endpoints publicos esenciales son `POST /auth/register`, `POST /auth/login`, rutas de recuperacion si estan habilitadas y `GET /health`. Los endpoints protegidos incluyen usuarios, clientes, mascotas, negocio, citas, servicios, planes, inventario, pagos, caja, facturas, notificaciones, documentos, autorizaciones, CRM, onboarding e identidad. Tambien existen modulos

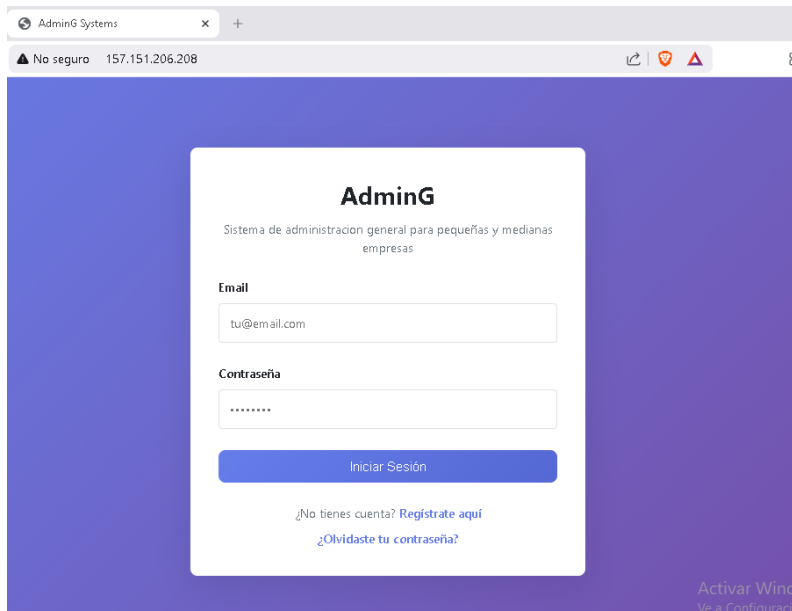
opcionales que pueden activarse segun configuracion: reportes, administracion, IA, operaciones, proyectos, tesoreria y otros dominios especializados.

En la evidencia visual se observa el login con campos de email y contrasena, boton de inicio de sesion, enlace de registro y recuperacion. Luego, el dashboard muestra barra lateral, plan Free, usuario `caniche1@example.com`, rol Manager, clientes totales, citas del dia, productos e ingresos del mes. Esa vista confirma que la autenticacion enlaza con datos operativos y reglas de plan.

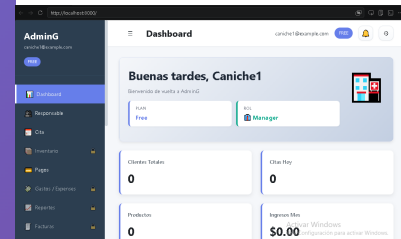
6.5 Evidencias graficas

6.5.1 Frontend en desarrollo y producción

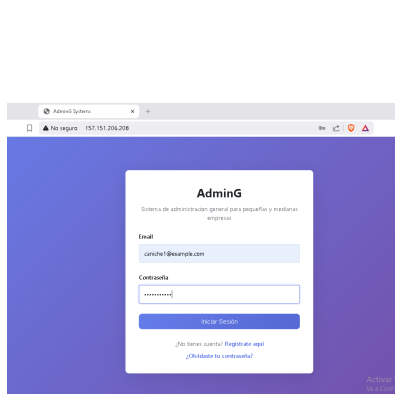
Las imagenes a continuacion son pruebas finales con rutas reales, en login y dashboard:



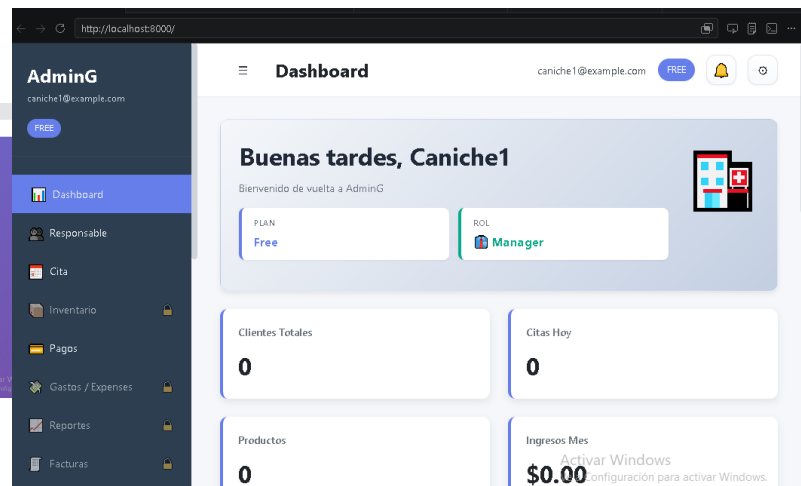
(a) Login en Oracle Free Tier



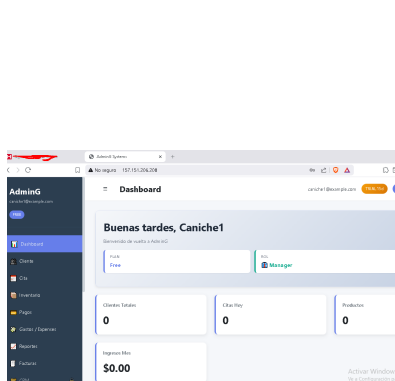
(a) Login local en VS Code



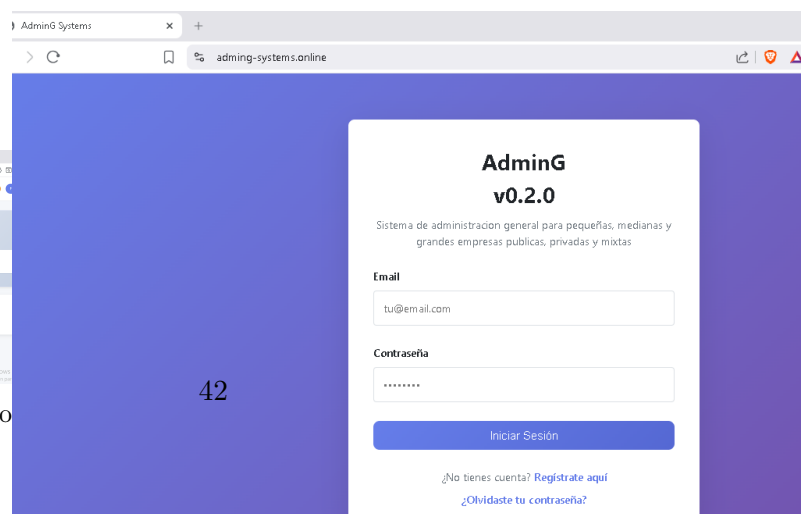
(a) Credenciales de prueba



(a) Dashboard local autenticado



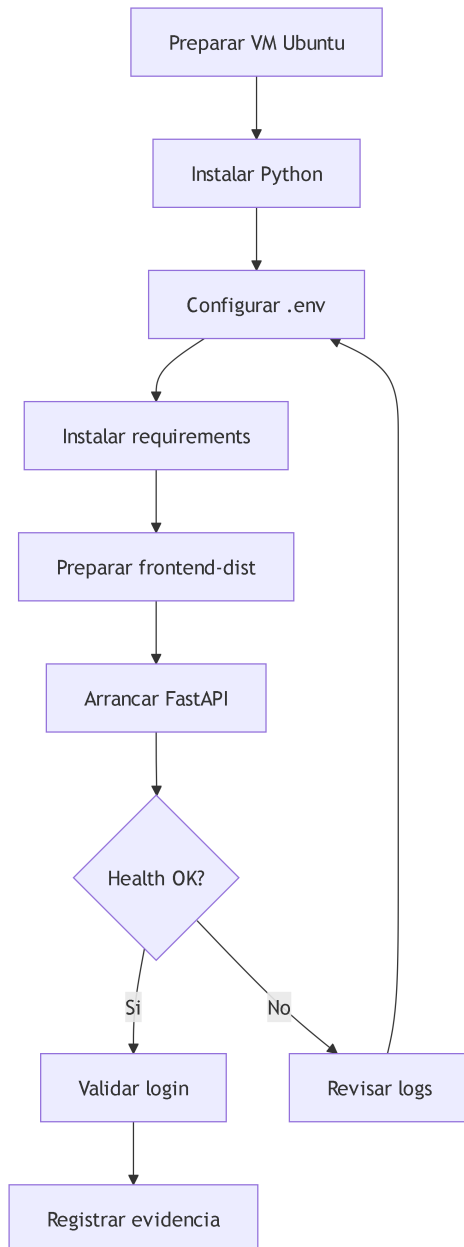
(a) Dashboard publico autenticado



(a) Login con dominio .online

6.5.2 Despliegue operativo

La operacion en la VM debe documentarse con comandos exactos. Una ruta prudente incluye crear usuario del sistema, instalar Python, clonar repositorio, crear entorno virtual, instalar `requirements.txt`, configurar `.env.production`, compilar o copiar `frontend-dist`, abrir puertos necesarios, validar `GET /health` e iniciar la aplicacion mediante `systemd` o un proceso controlado.



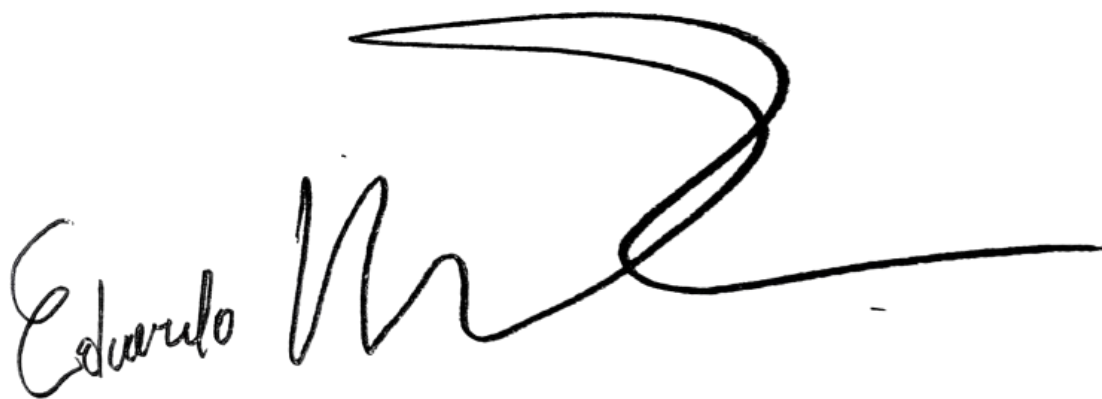
6.6 Cierre

La implementacion de AdminG ya cuenta con una base funcional y desplegada. El valor del Anexo E esta en convertir esa ejecucion en evidencia verificable: capturas, comandos, rutas, variables, pruebas y decisiones de operacion. Al mantener este anexo actualizado, el proyecto puede evolucionar sin perder trazabilidad entre lo planeado, lo construido y lo desplegado.

7 Anexo F

7.1 Firmas

7.1.1 Aprendiz



A handwritten signature in black ink. The name 'Eduardo' is written in a cursive script on the left. To its right is a large, stylized, and somewhat abstract signature mark that starts with a vertical stroke, loops around, and ends with a long horizontal tail.

7.1.2 Ente encargado

7.1.3 Instructor Sena de seguimiento
